

- [Efficient Fixed/Floating-Point Merged MAC Architecture](#)
 - [Signal Glossary](#)
 - [Key Design Notes](#)

Efficient Fixed/Floating-Point Merged MAC Architecture

Block diagram of the 3-stage pipelined, mixed-precision multiply-accumulate unit precisely mapped to the verified Verilog structural modules and exact signal widths.

Preview in VS Code or any Markdown viewer that supports Mermaid.

```
Parse error on line 6:
...olor:#000;    ACC([accumulator 32]):::
-----^
Expecting 'SPACE', 'GRAPH', 'DIR', 'TAGEND', 'TAGSTART',
'UP', 'DOWN', 'subgraph', 'end', 'PS', 'MINUS', '--', '==',
'STR', 'STYLE', 'LINKSTYLE', 'CLASSDEF', 'CLASS', 'CLICK',
'DEFAULT', 'NUM', 'PCT', 'COMMA', 'ALPHA', 'COLON', 'BRKT',
'DOT', 'PUNCTUATION', 'UNICODE_TEXT', 'PLUS', 'EQUALS',
'MULT', got 'SQS'
```

Signal Glossary

Signal	Width	Stage	Description
<code>eop_fp</code>	1	1	Effective operation: 1 = subtract ($sp \neq sc$), 0 = add
<code>shift_amt</code>	6	1	Right-shift to align accumulator: $ea + eb + 134 - ec$
<code>exp_align</code>	8	1 → 3	Exponent of bit 57 of the 58-bit field: $(ea+eb+134) \& 0xFF$
<code>c_mix</code>	58	1	FLP: aligned (possibly inverted) accumulator; FIX: zero-padded <code>addend_fix</code>
<code>comp</code>	1	1	Ones-complement fill flag from align_shifter

Signal	Width	Stage	Description
<code>stk</code>	1	1	Sticky bit: any set bits shifted out of the 58-bit window
<code>cin</code>	1	2	Carry-in to CPA: = <code>eop_fp</code> (completes ones-complement addition)
<code>sum_high</code>	27	2	Upper 27 bits of 58-bit sum: <code>cout ? inc_out : {1'b0, c_mix[57:32]}</code>
<code>is_neg</code>	1	2	<code>eop_fp & ~sum_high[26]</code> : subtraction with negative result → negate
<code>sign_fp_out</code>	1	2	Result sign: <code>is_neg ? ~sign_fp : sign_fp</code>
<code>sum_fp</code>	58	2	Post-complement 58-bit sum (passes through or 2's-complement negated)
<code>count</code>	6	2	LZA predicted shift (F-string, parallel with CPA); passed to Stage 3
<code>is_neg_r3</code>	1	2→3	<code>is_neg</code> pipelined into Stage 3 for normalization path selection
<code>actual_shift</code>	6	3	Exact LZC of <code>sum_norm_in</code> (ascending loop, highest set bit wins)
<code>sum_norm_out</code>	58	3	Left-shifted normalized mantissa
<code>exp_norm</code>	8	3	<code>exp_align - actual_shift</code>

Key Design Notes

Karatsuba split — FP16 11-bit mantissa: `mah` = `mant[10:8]` (3 bits), `mal` = `mant[7:0]` (8 bits). Product = `mal·mbl` + `mah·mbh·2^16` + `cross·2^8`. The cross term uses the same `mult8` hardware that handles INT8 in FIX mode.

58-bit accumulation field layout



[57:34]

↑ MSB of acc lands at 57 - shift_amt

[21:0]

Ones-complement subtraction — When `eop_fp=1`, the accumulator mantissa is bit-inverted by `invert_addend` and the lower 34 bits of `c_align` are filled with 1s. `cin=eop_fp=1` completes the two's complement across the CPA boundary. If `sum_high[26]` is not set (no end-around carry), the result is negative and `complementer` performs a true 2's-complement negation.

Exact LZC in normalization — The LZA F-string formula $F_pos[i] = P[i] \wedge Z[i+1]$ resolves carry structure top-down. For additions where operands occupy low-order bits, it always predicts `count ≈ 1` regardless of the true LZC (up to 57). `normalization_shifter` therefore performs an exact ascending-loop LZC on the post-complement `sum_norm_in` for all cases. The `count/valid` ports are forwarded for future use.